

EPL Prediction Market Trading Strategy

Technical Specification for Implementation

Quant Strategy Team

Draft v0.2 — August 29, 2026

Status	Draft v0.2 — for implementation by the quant dev team
Scope	Premier League football; season-long and single-match prediction market contracts
Objective	Trade the divergence between model probability p_{true} and market-implied probability p_{implied}

Contents

1 Purpose and Scope	3
2 Notation	3
3 Module 1 — Club Value Model	4
3.1 Player value	4
3.2 Manager value	5
3.3 Team base value	5
3.4 Contextual adjustments	5
3.5 Composite club value	6
4 Module 2 — Match Outcome Probability Model	6
4.1 From value differential to a distribution	6
4.2 Fitting m and σ	7
4.3 Heteroskedastic σ (v1.1 extension)	7
5 Module 3 — Season-Level Monte Carlo Simulation	8
5.1 Algorithm	8
5.2 Path dependency	9
5.3 Output	9
6 Module 4 — In-Play Value Model	9
6.1 Total in-play value	9
6.2 Time-decay of variance	10
6.3 Live probability	10
7 Module 5 — Data Requirements and Sourcing	11
8 Module 6 — Calibration and Backtesting	11
8.1 Historical dataset construction	11
8.2 Fitting procedure	11
8.3 Overfitting controls	12

9	Module 7 — Signal Generation and Execution	12
9.1	Mispricing signal	12
9.2	Position sizing and PnL realization	12
10	Module 8 — System Architecture	13
11	Summary of Explicit Judgment Calls	14

1 Purpose and Scope

The strategy trades outcome contracts on football prediction markets (e.g. “Team X finishes top 4,” “Team X beats Team Y ”) by:

- (1) estimating a model probability p_{true} for each contract from a bottom-up club-strength model;
- (2) comparing p_{true} to the market’s p_{implied} ;
- (3) sizing and executing trades when a sufficient edge exists, using a defined position-sizing and PnL-realization rule;
- (4) continuously recalibrating the model in-play as new information arrives (goals, substitutions, formation changes) for short-dated match contracts.

The document is organized into eight modules that map directly onto software components.

#	Module	Purpose
1	Club Value Model	Compute a scalar strength rating C for each club, pre-match
2	Match Outcome Model	Convert a value differential into win/draw/loss probabilities
3	Season Simulation Engine	Monte Carlo simulate the season to price top-4 / title / relegation markets
4	In-Play Value Model	Update club value in real time during a live match
5	Data Layer	Define required data and sourcing
6	Calibration & Back-testing	Fit and validate all model weights against history
7	Signal & Execution	Convert p_{true} vs. p_{implied} into sized trades and realized PnL
8	System Architecture	Compose the above into a production pipeline

Table 1: Module decomposition of the strategy.

A note on rigor: several formulas in the original strategy sketch (e.g. “integrate a normal distribution from $-\infty$ to the value difference”) are directionally correct but under-specified for implementation. This document tightens them into a standard, well-known model class — an **ordered probit** on a latent value differential — which is a direct, implementable generalization of “integrate the normal CDF.” Every place this happens is flagged explicitly (see Section 11) so the dev team knows where a judgment call was made.

2 Notation

All weights ($\lambda_{\bullet}$, decay rates β, γ, δ , the draw margin m , and σ) are **free parameters to be fit in Module 6**, not hand-set constants. Every formula below is a parametrized functional form, not a finished number.

Symbol	Meaning
X, Y	The two teams in a fixture (X is the team being priced “from”)
t	Match index / time index (context-dependent)
$V_{i,t}$	Value of player i heading into match t
$r_{i,t-k}$	Player i ’s performance rating k matches before t
$M_{j,t}$	Value of manager j heading into match t
C_X	Composite fundamental club value of team X (pre-match)
D	Value differential, $D = C_X - C_Y$
σ	Standard deviation of match-day noise not explained by D
$\Phi(\cdot)$	Standard normal CDF
λ_k	Learned weight for factor k
$p_{\text{true}}, p_{\text{implied}}$	Model probability and market-implied probability of an outcome
C_{ingame}	In-play adjustment to club value during a live match
C_{total}	$C_{\text{pre}} + C_{\text{ingame}}$, used for live win-probability updates

Table 2: Notation used throughout this document.

3 Module 1 — Club Value Model

3.1 Player value

Exponentially-weighted average of the player’s last $N = 10$ match ratings, with non-participation scored as zero:

$$r_{i,t-k} = \begin{cases} \text{rating of player } i \text{ in match } (t-k), & \text{if minutes played} \geq 10 \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

$$w_k = \beta^k, \quad \beta \in (0, 1) \quad (2)$$

$$V_{i,t} = \frac{\sum_{k=1}^{10} w_k r_{i,t-k}}{\sum_{k=1}^{10} w_k} \quad (3)$$

Implementation notes:

- Use a fixed-length rolling window per player, re-weighted every match rather than recomputed from scratch.
- β (≈ 0.85 as a starting point) is a single global hyperparameter initially; letting it vary by position (GK/DEF/MID/FWD) is a v2 extension, not v1 scope.
- Ratings must come from a standardized performance-rating provider so that “10.0 = best possible, 0 = did not play” is well-defined across positions (see Section 7).
- Injuries/suspensions are a *separate* subtractive term (Section 3.4), not a zeroing of $V_{i,t}$: a player injured for the *upcoming* match still has a valid historical V , which is what must be removed from the team sum.

3.2 Manager value

Same exponential-decay structure over the manager’s last 5 matches, using a team-level performance score $s_{j,t-k}$ attributable to the manager (e.g. points-per-game relative to a pre-season baseline):

$$M_{j,t} = \frac{\sum_{k=1}^5 \gamma^k s_{j,t-k}}{\sum_{k=1}^5 \gamma^k} \quad (4)$$

where γ is a decay hyperparameter fit independently of β .

3.3 Team base value

$$C_{\text{base},X,t} = \sum_{i \in \text{XI}(X,t)} V_{i,t} + M_{j,t} \quad (5)$$

The composition of $\text{XI}(X,t)$ is a key implementation decision, since the true starting eleven is unknown pre-kickoff. Two supported approaches:

- (1) **Expected-minutes weighting (recommended default).** Maintain a rolling probability-of-starting weight $\theta_{i,t} \in [0, 1]$ for every squad player, and use

$$C_{\text{base},X,t} = \sum_{i \in \text{squad}(X)} \theta_{i,t} V_{i,t} + M_{j,t} \quad (6)$$

This avoids a brittle discrete lineup prediction and degrades gracefully far from kickoff.

- (2) **Confirmed-XI override.** Once official lineups are published (~ 60 minutes pre-kickoff), switch to the discrete sum over the confirmed 11 players. This strictly more accurate, late-arriving signal should *override* (1) via a data-freshness gate, not be averaged with it.

3.4 Contextual adjustments

Each adjustment is a signed additive term on C_{base} , with coefficients fit rather than assumed.

- (a) **Team news (injuries / suspensions / red cards).**

$$C_{\text{news},X,t} = - \sum_{i \in \text{unavailable}(X,t)} \theta_{i,t}^{\text{baseline}} V_{i,t} \quad (7)$$

Subtract the expected-minutes-weighted value of every unavailable player. Under approach (1) this is largely automatic (set $\theta_{i,t} = 0$); keep it as an explicit, auditable term regardless.

- (b) **Home / away.**

$$C_{\text{venue},X,t} = \begin{cases} +\lambda_{\text{home}} & \text{if } X \text{ is home} \\ -\lambda_{\text{away}} & \text{if } X \text{ is away} \end{cases} \quad (8)$$

A learned constant in the same units as C (not a percentage); do not assume stability across seasons — crowd and travel effects drift, so re-fit periodically (Section 8).

(c) Head-to-head.

$$C_{\text{h2h},X,t} = \sum_{k=1}^K \delta^k h_{X,t-k} \quad (9)$$

where $h_{X,t-k}$ is a signed result-strength score for the k -th most recent meeting between X and Y specifically, δ is a decay parameter fit separately from β, γ , and K is capped (e.g. last 10 meetings or 5 years, whichever is fewer).

(d) Table position / matches remaining.

$$C_{\text{table},X,t} = f(\text{position}_X, \text{points_gap_to_target}, \text{matches_remaining}) \quad (10)$$

Matters most for season-long contracts (Module 3); included here as a small context/bias term (dead-rubber effects, must-win desperation) rather than a leading factor. Do not hard-code the functional form of f until backtesting indicates its shape.

(e) Transfer market. Adjust squad inputs directly rather than as a bolt-on term:

$$V_{\text{new player},t} = \frac{1}{5} \sum_{k=1}^5 \tilde{r}_{\text{new player},t-k} \quad (11)$$

where $\tilde{r}$ is the player's rating on a *league-normalized* scale from their previous club/competition. New signings with no historical rating receive a provisional V from a market-value proxy (Section 7) until 5 top-flight performances accumulate, and should carry elevated uncertainty (Section 4.3), not just an adjusted C .

3.5 Composite club value

$$C_{X,t} = C_{\text{base},X,t} + C_{\text{news},X,t} + C_{\text{venue},X,t} + C_{\text{h2h},X,t} + C_{\text{table},X,t} \quad (12)$$

(Transfer-market effects are already embedded in C_{base} via updated V_i .) All additive terms should be stored as **separate fields**, not pre-summed, so Module 6 can attribute backtest error to individual components.

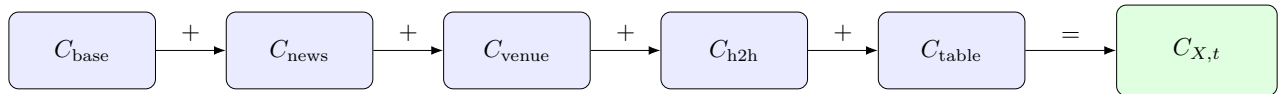


Figure 1: Composition of the club value $C_{X,t}$ from Equation (12). Each component is stored independently in the feature store for auditability and re-weighting.

4 Module 2 — Match Outcome Probability Model**4.1 From value differential to a distribution**

Define the pre-match value differential $D = C_X - C_Y$. The original spec calls for “a normal distribution, integrated from $-\infty$ to D ” — taken literally this only yields a two-outcome (win / not-win) probability. Football markets need three outcomes, so this is formalized as an **ordered probit**: let $m > 0$ be a fitted *draw margin* and σ the fitted match-day noise standard deviation, and model the realized result via a latent variable $L \sim \mathcal{N}(D, \sigma^2)$, with

$$\text{result} = \begin{cases} X \text{ loses,} & L < -m \\ \text{draw,} & -m \leq L \leq m \\ X \text{ wins,} & L > m \end{cases} \quad (13)$$

giving

$$P(X \text{ wins}) = 1 - \Phi\left(\frac{m - D}{\sigma}\right) \quad (14)$$

$$P(\text{draw}) = \Phi\left(\frac{m - D}{\sigma}\right) - \Phi\left(\frac{-m - D}{\sigma}\right) \quad (15)$$

$$P(X \text{ loses}) = \Phi\left(\frac{-m - D}{\sigma}\right) \quad (16)$$

By construction $P(\text{win}) + P(\text{draw}) + P(\text{loss}) = 1$, which is not guaranteed by an ad hoc hand-rolled three-way split. As $m \rightarrow 0$ this collapses to the originally-described “integrate to D ” two-outcome formula, so Equations (14)–(16) are a strict superset of the requested behaviour.

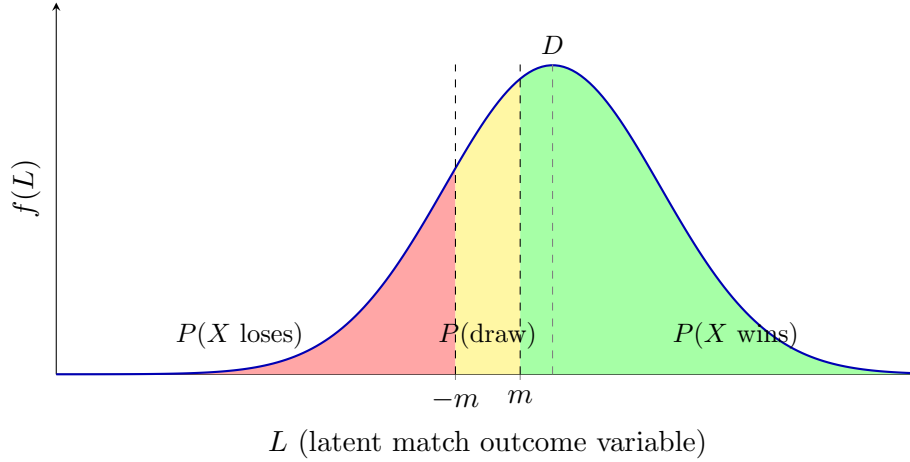


Figure 2: Ordered probit model: $L \sim \mathcal{N}(D, \sigma^2)$ is split into loss / draw / win regions by the draw margin $\pm m$. Shown for illustrative values $D = 0.6$, $m = 0.3$, $\sigma = 1$.

4.2 Fitting m and σ

Both are single global scalars to start (do not add team- or fixture-specific σ until v1 is validated), fit by maximum likelihood on historical (D, result) pairs — see Section 8. Do not hand-pick these values.

4.3 Heteroskedastic σ (v1.1 extension)

σ should shrink when more is known about a fixture (confirmed lineups, no transfer-window uncertainty) and grow when key inputs are unusually uncertain (new signing with no top-flight track record, first-ever meeting, manager in first 5 games):

$$\sigma_t = \sigma_0 \exp\left(\sum_k \kappa_k u_{k,t}\right) \quad (17)$$

where $u_{k,t}$ are uncertainty-indicator features. Not required for a working v1, but σ should be implemented as a function call, not a hardcoded constant, even in v1, so this extension is a drop-in change.

5 Module 3 — Season-Level Monte Carlo Simulation

Used to price contracts such as “Chelsea finishes top 4,” “Team X wins the title,” “Team X is relegated.”

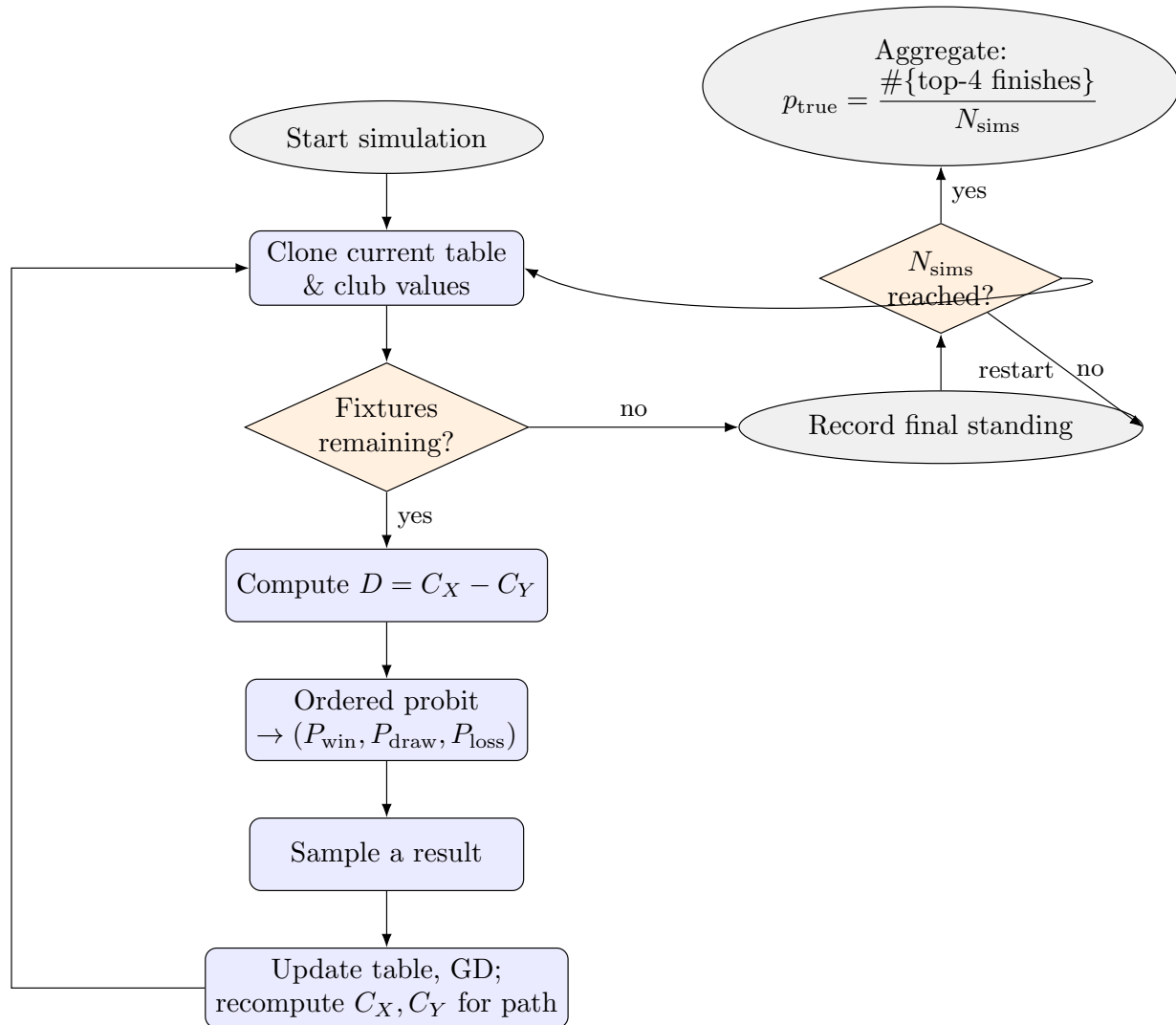


Figure 3: Season Monte Carlo simulation loop. Club values are recomputed *within* each simulated path (Section 5.2), which is the central subtlety of this module.

5.1 Algorithm

```

for sim in 1..N_SIMS:                                # e.g. N_SIMS = 100,000
    state = clone(current_table_and_club_values)
    for (X, Y) in remaining_fixtures_in_date_order:

```

```

D = state.C[X] - state.C[Y]
(p_win, p_draw, p_loss) = ordered_probit(D, m, sigma)
result = sample_categorical([p_win, p_draw, p_loss])
update state.table with result (points, GD, GF)
update state.C[X], state.C[Y]          # path-dependent, see 5.2
record final_standing(sim)

```

```
p_true(top4) = ( #{sim : team finishes rank 1-4} ) / N_SIMS
```

5.2 Path dependency

Because V_i and C_{h2h} are functions of *recent* match results, a team’s club value at gameweek 30 within a given simulated path depends on that path’s simulated gameweeks 20–29, which differ across paths. Consequences:

- A single static C_X cannot be pre-computed for the rest of the season and reused across simulations — it must be recomputed *within* each path as results are drawn.
- For tractability, simulate at the level of match result (W/D/L, or a simple goal-difference draw) rather than full player-level match events, feeding the simulated result back into a simplified team-level “implied performance rating” update rather than resimulating individual players. This is the single largest approximation in this module and should be flagged as such in code.
- Player availability for *future* simulated fixtures should be modelled probabilistically (base injury hazard rate, deterministic suspension triggers from simulated cards) rather than frozen at today’s team news, otherwise run-in variance is understated.

5.3 Output

Per team, per simulation batch: the full distribution over final league position, and point estimates with confidence intervals for any threshold market (top 4, top 6, bottom 3). Re-run nightly *or* after any material news event (red card, injury, notable result) rather than only once per gameweek, since p_{implied} moves continuously.

6 Module 4 — In-Play Value Model

For “will X win this currently-in-progress match” contracts.

6.1 Total in-play value

$$C_{\text{total},X}(t) = C_{\text{pre},X} + C_{\text{ingame},X}(t) \quad (18)$$

$C_{\text{pre},X}$ is the Module 1 output frozen at kickoff. The in-game term accumulates live:

$$C_{\text{ingame},X}(t) = \lambda_{\text{score}}(G_X(t) - G_Y(t)) + \lambda_{\text{form}} \cdot \text{Form}_X(t) + \sum_{\text{subs}} \left[\lambda_{\text{swap}}(V_{\text{on}} - V_{\text{off}}) + \lambda_{\text{tac}} \cdot \text{Tac}(\text{sub context}) \right] \quad (19)$$

- **Scoreline:** signed goal difference *at time t* , recomputed on every goal. This term needs a time-remaining interaction (a 1-goal lead at minute 85 is worth far more than at minute 10) — handled via $\sigma(t)$ below, not by making λ_{score} itself time-varying.
- **Formation (Form $_X(t)$):** an empirically-fit, signed feature of formation shape-change conditioned on game state (e.g. going more defensive while leading). Do not hand-sign this term — “sit deep when leading” does not universally help; let backtesting determine sign and magnitude.
- **Substitutions:** $V_{\text{on}}, V_{\text{off}}$ reuse the Module 1 player-value definition; $\text{Tac}(\cdot)$ is a smaller learned term capturing sub *timing* (minute, game state at substitution).
- **H2H and home/away** are already embedded in $C_{\text{pre},X}$ and are not re-derived here.

6.2 Time-decay of variance

As the clock runs down, outcome uncertainty shrinks mechanically, independent of C_{ingame} :

$$\sigma(t) = \sigma_0 \cdot g(\tau), \quad g(\tau) = \sqrt{\tau/90} \quad (20)$$

where τ is minutes remaining (g to be re-fit against realized in-play frequencies; the square-root form is a starting functional form, analogous to time-decay in other sports’ live win-probability models). This term is **required**, not optional: without it the model will be systematically miscalibrated either early or late in matches.

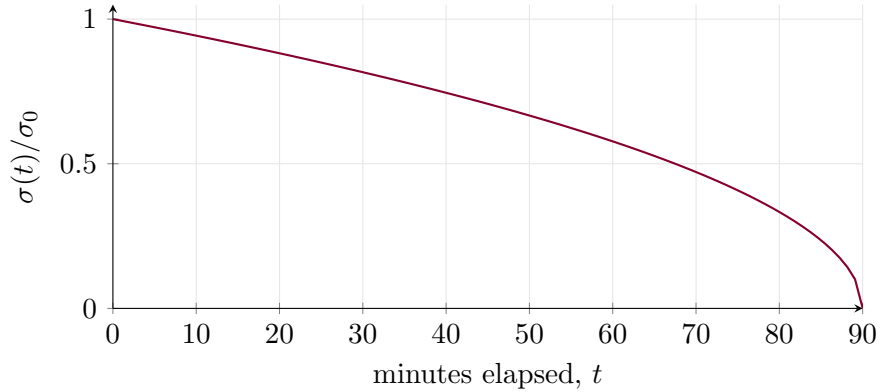


Figure 4: Illustrative time-decay of in-play match-outcome variance, $g(\tau) = \sqrt{\tau/90}$, as minutes elapse. Uncertainty shrinks toward full time regardless of scoreline.

6.3 Live probability

Re-apply Equations (14)–(16) with $D \rightarrow C_{\text{total},X}(t) - C_{\text{total},Y}(t)$ and $\sigma \rightarrow \sigma(t)$:

$$P(X \text{ wins} \mid t) = 1 - \Phi\left(\frac{m - D_{\text{total}}(t)}{\sigma(t)}\right) \quad (21)$$

Recompute on every scoreline change, substitution, and formation change, plus on a fixed heartbeat (e.g. every 60 seconds) to capture pure time decay.

Data	Used in	Candidate sources
Player ratings per match	§3.1	Licensed standardized-rating provider (Opta/StatsPerform, WhoScored, SofaScore feed)
Minutes played per player	§3.1	Same provider, or official match-centre data
Manager tenure & results	§3.2	League results feed + manager-change dataset
Table, points, GD, fixtures remaining	§3.4(d), §5	Official league API (e.g. football-data.org)
Historical head-to-head results	§3.4(c)	Historical results archive (e.g. football-data.co.uk)
Injuries / suspensions / team news	§3.4(a)	Dedicated injury feed (PhysioRoom, Premier Injuries) + press-conference confirmation
Transfer activity & valuations	§3.4(e)	Transfermarkt (prior for unrated players) + rating provider for pre-move history
Live scoreline, formation, subs, cards	§6	Low-latency live feed (Sportradar, Genius Sports, Opta live)
Market-implied price p_{implied}	§9	Prediction market order book / last-traded-price API

Table 3: Data requirements by module.

7 Module 5 — Data Requirements and Sourcing

Two points for the dev team specifically: (1) **rating-scale normalization** must happen at the ingestion boundary — do not let a provider’s raw 1–10 scale enter Section 3.1 unnormalized; (2) **latency tiers differ by module** — pre-match modules (1–3) can run on an hourly/nightly batch cadence, while the in-play module (4) needs streaming, event-driven ingestion. Building these as one monolithic pipeline will make the in-play latency requirement bottleneck the season-simulation batch job, or vice versa.

8 Module 6 — Calibration and Backtesting

This module turns every $\lambda_{\bullet}, \beta, \gamma, \delta, m, \sigma$ above from a placeholder into a fitted number.

8.1 Historical dataset construction

Build a match-level dataset spanning as many prior seasons as data availability allows (target ≥ 8 –10 seasons). For every historical match, reconstruct C_X, C_Y , and every sub-component **using only information available before kickoff** — implement as a strict as-of join by timestamp. Any leakage of post-match information into pre-match features will produce a model that looks strong historically and fails live.

8.2 Fitting procedure

Fit Module 1 weights and Module 2 parameters (m, σ) jointly by maximum likelihood: choose the parameter vector $\Theta = (\beta, \gamma, \delta, \lambda_{\text{home}}, \lambda_{\text{away}}, \dots, m, \sigma)$ that maximizes the log-likelihood of the observed historical result sequence,

$$\hat{\Theta} = \arg \max_{\Theta} \sum_{\text{matches}} \log P_{\Theta}(\text{observed result} \mid D) \quad (22)$$

using a gradient-based optimizer (L-BFGS or Adam over a differentiable implementation) once past ~ 4 –5 parameters.

Metrics to track (not only optimize on):

$$\text{Log-loss} = -\frac{1}{n} \sum_{j=1}^n \log \hat{p}_{j,y_j} \quad (23)$$

$$\text{Brier score} = \frac{1}{n} \sum_{j=1}^n \sum_{c \in \{W,D,L\}} (\hat{p}_{j,c} - \mathbb{I}[y_j = c])^2 \quad (24)$$

plus a **calibration / reliability curve** (bucket predicted probabilities into deciles, plot against realized frequency) — the single most important diagnostic, since a model can post a good average Brier score while being badly miscalibrated at the tails, which is exactly where this strategy trades.

Validation scheme: walk-forward / rolling-origin only, never a random train/test split. Fit on seasons 1.. k , validate on season $k+1$, roll forward; report out-of-sample metrics per validation season so regime shifts (rule changes, VAR introduction, etc.) remain visible.

Module 3 (season simulation) is validated by comparing simulated “probability of top 4 as of gameweek g ” against realized outcomes pooled across many historical team-seasons. Module 4 (in-play) is validated by bucketing (D_{total}, τ) pairs and checking realized win frequency against predicted, across many historical matches with full event timelines.

8.3 Overfitting controls

L2-regularize the weight vector given the number of free parameters relative to matches per season (~ 380); hold out the most recent 1–2 seasons as a never-touched final test set; re-fit on a fixed schedule (e.g. each close season) rather than continuously online, logging every re-fit’s parameters and out-of-sample metrics for audit.

9 Module 7 — Signal Generation and Execution**9.1 Mispricing signal**

For every open contract:

$$\text{edge} = p_{\text{true}} - p_{\text{implied}} \quad (25)$$

Only proceed to sizing if $|\text{edge}|$ exceeds a minimum threshold sized to cover expected transaction costs and spread — do not trade every non-zero edge.

9.2 Position sizing and PnL realization

Using the original theory sheet’s notation, with k a position-sizing scale constant:

- If $p_{\text{true}} < p_{\text{implied}}$: buy $k(p_{\text{implied}} - p_{\text{true}})^2$ **No** shares. When the market subsequently converges so that $p_{\text{implied}} = p_{\text{true}}$, sell those shares; realized PnL on that leg is

$$\text{PnL} = k(p_{\text{implied}}^{\text{entry}} - p_{\text{true}})^3 \quad (26)$$

- If $p_{\text{true}} > p_{\text{implied}}$: buy $k(p_{\text{true}} - p_{\text{implied}})^2$ **Yes** shares, with the symmetric convergence/exit logic.

Implementation flags for the dev team:

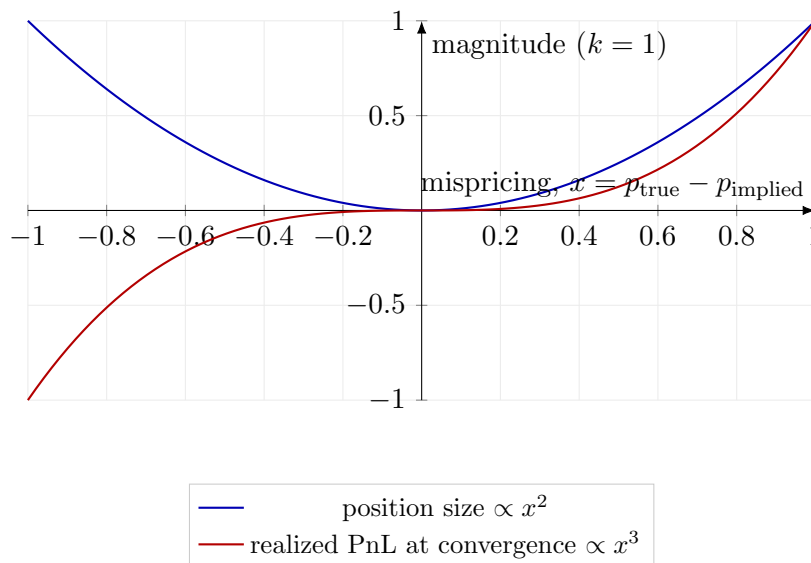


Figure 5: Sizing is quadratic (symmetric, direction-agnostic) in the mispricing; realized PnL at convergence is cubic (sign-preserving, accelerating for large edges). Small edges are sized conservatively; large edges aggressively.

1. This sizing rule is *not* Kelly-optimal in the classical sense — it behaves like a market-maker heuristic that only sizes up when confident. Backtest sensitivity to k explicitly, and impose a hard max-position-per-contract and max-aggregate-exposure-per-team/gameweek independent of the formula, since the formula alone has no notion of bankroll or liquidity.
2. The PnL formula assumes convergence occurs before resolution. In practice: (a) if the contract resolves first, settlement PnL must be computed from the actual outcome and entry price, not the convergence formula; (b) p_{true} itself moves as news arrives, so treat it as “current model estimate,” not a fixed target, and re-derive continuously. Build exit logic as: exit on convergence-to-current-estimate **or** on resolution **or** on a stop-loss if the gap widens and stays wide beyond a re-validation window (suggesting the model, not the market, is wrong) — this stop-loss condition is an addition to the original sketch.
3. Subtract expected round-trip transaction costs from both the entry threshold (§9.1) and backtest PnL, or live performance will be overstated.
4. Scale k down when the underlying p_{true} estimate has wide Monte Carlo standard error (a natural Module 3 output) or when σ_t indicates an unusually uncertain fixture — two contracts with equal nominal edge should not necessarily receive equal size.

10 Module 8 — System Architecture

Key non-functional requirements: (1) **idempotent, replayable feature computation** — the same feature-computation code path must run in both backtest and live modes; (2) **separate deploy/scale cadence** for batch pre-match modules vs. the streaming in-play module; (3) **full audit logging** of every p_{true} estimate, its input features, and model/parameter version, tied to every trade.

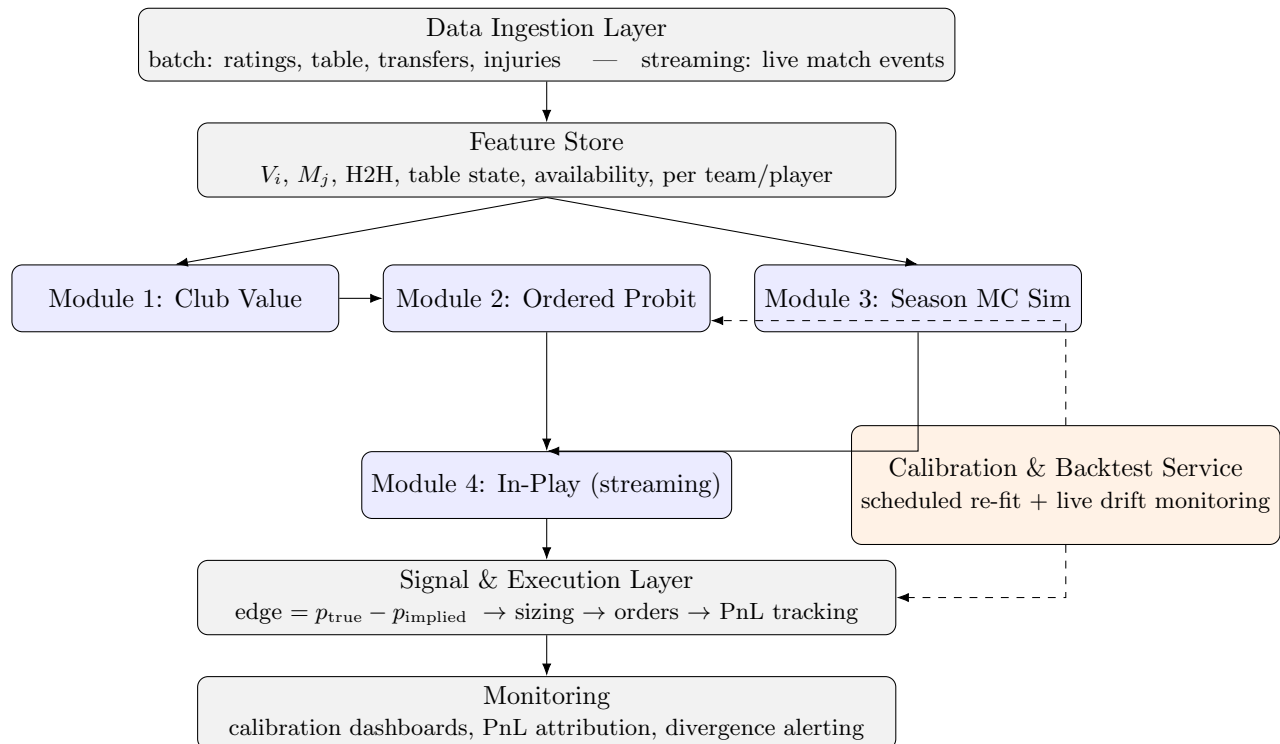


Figure 6: System architecture: batch and streaming ingestion feed a shared feature store; Modules 1–4 compose into the signal layer; the calibration service periodically re-fits weights and monitors live calibration drift.

11 Summary of Explicit Judgment Calls

For traceability, since this specification formalizes under-specified parts of the original sketch:

1. Win/draw/loss modelled as an **ordered probit** (draw margin m + normal noise σ) rather than a bare normal-CDF integral, so three outcomes are handled and probabilities sum to 1.
2. Starting XI handled via **expected-minutes weighting** with a confirmed-lineup override, rather than a discrete lineup prediction.
3. The season Monte Carlo simulation is explicitly **path-dependent** (club values evolve within each simulated path) — flagged as the largest source of approximation, since full player-level in-path resimulation is likely intractable at 10^5 -simulation scale.
4. The in-play model requires a **time-decaying variance** $\sigma(t)$, absent from the original sketch, without which live probabilities will be systematically miscalibrated.
5. Added a **stop-loss / model-disagreement** exit condition and a **transaction-cost floor** to execution logic, since the original PnL formulas assume convergence and ignore costs.

Everything else — the player-value decay structure, manager value, H2H decay, home/away constant, transfer-market handling, and the quadratic-sizing/cubic-PnL rule — is implemented as originally specified, with hyperparameters left explicitly open for Module 6 to fit rather than hard-coded.